❽ Maps the Razor Pages endpoints

In this example, the `IWebHostEnvironment` object is injected into the `Configure()` method using DI so that you can configure the middleware pipeline differently in development and production. In this case, we add the `DeveloperExceptionPageMiddleware` to the pipeline when we're running in development.

NOTE Remember that `WebApplication` adds this middleware automatically, but with `Startup` you must add it manually. The same goes for all the other automatically added middleware.

After adding all the middleware to the pipeline, you come to the `UseEndpoints()` call, which adds the `EndpointMiddleware` to the pipeline. When you use `WebApplication`, you rarely need to call this, as `WebApplication` automatically adds it at the end of the pipeline, but when you use `Startup`, you should add it at the end of your pipeline.

Note as well that the call to `UseEndpoints()` is where you define all the endpoints in your application. Whether they're Razor Pages, Model-View-Controller (MVC) controllers, or minimal APIs, you must register them in the `UseEndpoints()` lambda.

NOTE Endpoints must be registered inside the call to `UseEndpoints()` using the `IEndpointRouteBuilder` instance from the lambda method.

Other than the noted differences, moving your service, middleware, and endpoint configuration between a `Startup`-based approach and `WebApplication` should be relatively simple, which may lead you to